

Gebeya Week 1 Challenge

Slack Messages Analysis — Complete Project Guide

Biniam Tsige · Batch 6 · Data Engineering & ML Engineering
Generated: June 01, 2026

This document is a complete record of everything built, learned, and debugged during the Gebeya Week 1 Challenge. It covers all 5 tasks — from raw data loading to cloud deployment — including every concept explained, every question asked, every mistake made, and every fix applied.

Task	Topic	Status
Task 1	Git Setup & Exploratory Data Analysis	Complete
Task 2	ML Analysis (LDA, Sentiment, Classification)	Complete
Task 3	Databases (MongoDB + PostgreSQL)	Complete
Task 4	Streamlit Dashboard + Docker	Complete
Task 5	Infrastructure (Terraform + AWS + CI/CD)	Complete
Bonus	MongoDB Atlas + Streamlit Cloud Deployment	Complete

1. Development Environment Setup

Before writing any code, the development environment was set up correctly. This is one of the most important steps — the wrong setup causes mysterious errors that have nothing to do with your code.

1.1 Virtual Environment (gebvenv)

A virtual environment is an isolated Python workspace. Think of it like a separate room in your house dedicated to this project. Everything installed inside it stays inside it and doesn't affect any other Python project on your Mac.

Why this matters: if you install pandas for one project and it conflicts with an older version needed by another project, your entire Python setup breaks. Virtual environments prevent this completely.

```
# Create the virtual environment
python3 -m venv gebvenv

# Activate it (must do this every session)
source gebvenv/bin/activate

# Your terminal prompt changes to:
(gebvenv) btsm@MacBook Gebeya-Tech %
```

The virtual environment is named **gebvenv** — not the default 'venv'. This is intentional to make it clearly identifiable as this project's environment.

1.2 Homebrew — Mac Package Manager

Homebrew is the package manager for macOS system tools. Think of it like the App Store, but for developer tools that don't have a graphical installer. You use it to install things like databases, infrastructure tools, and runtimes.

Tool	Installed via	Purpose
MongoDB	brew install mongodb-community	NoSQL database for raw data storage
PostgreSQL	brew install postgresql@15	SQL database for analytics
Terraform	brew install hashicorp/tap/terraform	Infrastructure as Code
Docker	Download from docker.com	Container platform

Homebrew installs tools **globally** on your Mac — available to all projects and in any terminal. This is correct for system tools. Python packages, however, should be installed inside gebvenv using pip.

```
# Check what Homebrew has installed
brew list

# Check running services
brew services list
```

1.3 pip — Python Package Manager

pip installs Python libraries inside your virtual environment. Always activate gebvenv first, then use pip — otherwise packages install to the wrong location.

Package	Purpose
pandas	Data manipulation — DataFrames, filtering, grouping
matplotlib / seaborn	Chart and graph generation
scikit-learn	LDA topic modelling (LatentDirichletAllocation)
textblob	Pre-trained sentiment analysis
nltk	Natural language processing (stopwords, tokenization)
mlflow	Experiment tracking and model versioning
pymongo	Python driver for MongoDB
psycopg2-binary	Python driver for PostgreSQL
streamlit	Interactive web dashboard framework
pytest	Unit testing framework
reportlab	PDF generation
Pillow	Image loading for dashboard charts

1.4 Jupyter Notebooks & Kernel Registration

Jupyter notebooks run code in cells interactively. The key issue on this project: Jupyter must use the gebvenv kernel, not the system Python. If the wrong kernel is selected, all your installed packages are invisible and you get ModuleNotFoundError.

```
# Register gebvenv as a Jupyter kernel
python -m ipykernel install --user --name=gebvenv --display-name='Python (gebvenv)'

# When opening a notebook, always select:
# Kernel → 'Python (gebvenv)'
```

When you save a Jupyter notebook (.ipynb), it saves both the code AND the output (charts, tables, printed results). The notebook is a JSON file containing all of this together.

2. Task 1 — Git Setup & Exploratory Data Analysis

Task 1 had two goals: set up proper version control with Git and GitHub, and perform exploratory data analysis (EDA) on the Slack messages dataset.

2.1 The Dataset

The dataset is a Slack export from the Gebeya community workspace — messages from 39 channels spanning multiple weeks. The data is in JSON format, organized as one file per channel per day.

Fact	Value
Total messages loaded	20,011
Channels	39
Format	JSON files in anonymized/ folder
Privacy rule	NEVER push anonymized/ to GitHub

The data is anonymized — user IDs appear as codes like U03T89ACUUW instead of real names. @mention tags were also stripped during anonymization, which is why the mention count analysis showed near-zero values for all users.

2.2 SlackDataLoader Class

Instead of writing raw file-reading code everywhere, we built a class called `SlackDataLoader` that encapsulates all data loading logic. A class is a blueprint that groups related functions (methods) together.

```
class SlackDataLoader:
    def __init__(self, data_path=DATA_PATH, source='auto'):
        self.data_path = data_path
        self.source = 'mongo' if MONGO_URI else 'local'

    def get_channels(self):      # returns list of channel names
    def load_channel(channel):  # loads one channel as DataFrame
    def load_all_channels():    # loads all channels combined
```

Why use a class instead of plain functions? Because `data_path` is shared across all three methods. Setting it once in `__init__` means all methods automatically have access via `self.data_path` — no need to pass it every time.

2.3 What is a pandas DataFrame?

A `DataFrame` is like a spreadsheet in Python — rows and columns. When we load Slack messages, each row is one message and columns are its properties:

channel	user	text	ts	type	reply_count
ai-community-building	U03T89AC	Has anyone...	1234567890.1	message	5

all-week3	U03TEPYRM	Yes, you need...	1234567891.2	message	0
-----------	-----------	------------------	--------------	---------	---

2.4 The Four Analysis Dimensions

For each user, we measured four things. These measure different aspects of how a person participates in the community:

Metric	What it measures	Top user = ...
Message count	How many messages they posted	Most active poster
Reply count	How many replies their posts received	Best conversation starter
Reaction count	How many emoji reactions their posts got	Most appreciated poster
Mention count	How often they @mentioned others	Most connected (showed 0 — anonymized)

The key insight: these metrics tell very different stories. A user can post constantly (high message count) but get no reactions (low reaction count). Another user might post rarely but spark huge discussions (low messages, high replies).

2.5 Channel Activity Scatter Plot

The scatter plot showed each channel as a dot with: X = number of messages, Y = total replies + reactions. The result was a classic long-tail distribution.

ai-community-building was a massive outlier at ~8,500 messages and ~11,000 engagements — far beyond all other channels. This single channel drove the majority of all community activity.

2.6 Reply Time Analysis

We measured the time between a message being posted and receiving its first reply, then plotted against the hour of day. Findings:

- Median reply time: 0.6 minutes — most replies came within seconds
- Community was highly responsive during active hours
- Messages posted late at night waited hours — sometimes 1,400+ minutes
- This shows a natural active window vs off-hours pattern

2.7 CI/CD Workflows (GitHub Actions)

Three automatic checks run every time code is pushed to GitHub:

Workflow	File	What it checks	Passes when
Flake8	flake8_check.yml	PEP 8 code style	No style violations
Docstrings	docstring_tests.yml	Every function documented	All have docstrings
Unit Tests	unittests.yml	Code correctness	All 5 pytest tests pass

Flake8 enforces PEP 8 — Python's official style guide. Rules include: max 100 characters per line, no unused imports, proper blank lines between functions. Several fixes were needed in `dashboard.py` and `test_loader.py` to make it pass.

3. Task 2 — ML Analysis

Task 2 applied machine learning and statistical analysis to the Slack messages. Three different techniques were used, each serving a different purpose.

3.1 Time Difference Histograms

A histogram shows how frequently values fall into ranges (called bins). We measured the time gap between consecutive events — how quickly the next message, reply, or reaction appeared after the previous one.

Chart	Median gap	What it tells us
Consecutive Messages	0.6 min	Messages come in rapid bursts
Consecutive Replies	0.8 min	Thread replies also happen fast
Consecutive Reactions	1.4 min	Reactions slightly slower — reading before clicking
All Events combined	0.6 min	Overall community rhythm

The x-axis was capped at 120 minutes. All gaps longer than 120 minutes were grouped into the last bar — this is why there appears to be a spike at 120, it actually means '120 minutes or more'.

3.2 Rule-Based Message Classification

Before applying ML, we classified messages using hand-written rules. This is not machine learning — it's if/else logic written by a human:

```
def classify_message(text):
    tech = any(keyword in text.lower() for keyword in TECH_KEYWORDS)
    if re.search(r'\?|how|what|why', text.lower()):
        return 'Question-Technical' if tech else 'Question-NonTechnical'
    if re.search(r'^(yes|no|sure|correct)', text.lower()):
        return 'Answer'
    return 'Comment-Technical' if tech else 'Comment-NonTechnical'
```

TECH_KEYWORDS included: error, bug, code, python, git, pandas, model, docker, api, sql, jupyter, and others. Any message containing these words was classified as technical.

3.3 LDA Topic Modelling — Real Model Training

LDA (Latent Dirichlet Allocation) is an unsupervised machine learning algorithm that discovers hidden topics in a collection of text. 'Unsupervised' means we never told it what the topics were — it found them on its own.

How it works:

- Step 1 — CountVectorizer converts all messages into a matrix of word counts. Each row is a message, each column is a word, each cell is how many times that word appears.

- Step 2 — LDA analyzes which words appear together frequently across messages. Words that co-occur often are grouped into a topic.
- Step 3 — The model outputs topics as word distributions. Topic 1 might be 'python, error, install, pip' and Topic 2 might be 'week, task, submit, deadline'.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

vectorizer = CountVectorizer(stop_words='english', max_features=1000)
X = vectorizer.fit_transform(messages) # ← builds vocabulary from our data

lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(X) # ← THIS is the actual model training
```

The `.fit()` call is model training — the LDA model learns patterns from our specific Slack messages. This is the only genuine model training in the entire project.

3.4 Sentiment Analysis with TextBlob

Sentiment analysis measures whether text is positive, negative, or neutral. TextBlob assigns a polarity score between -1.0 (very negative) and +1.0 (very positive). A score near 0 is neutral.

```
from textblob import TextBlob

score = TextBlob('Great work everyone!').sentiment.polarity
# score = 0.8 (positive)

score = TextBlob('This is broken and nothing works').sentiment.polarity
# score = -0.4 (negative)
```

TextBlob uses a pre-trained lexicon — a dictionary of words labeled as positive or negative by researchers. We did NOT train this model. We just applied it to our messages. We then grouped scores by day and plotted the trend over time using a Savitzky-Golay smoothing filter.

3.5 MLFlow — Experiment Tracking

MLFlow is a tool that logs every time you train a model — recording the parameters used, the results produced, and saving the model itself. Think of it like a lab notebook for machine learning experiments.

MLFlow concept	What it means
Experiment	A named group of runs (e.g. 'LDA Topic Modelling')
Run	One execution of training with specific parameters
Parameters	Settings used (e.g. <code>n_topics=5</code> , <code>max_features=1000</code>)
Artifacts	Files saved (model file, vectorizer file, charts)
Model registry	Versioned storage of trained models

3.6 The SSL Certificate Issue (and the fix)

When running NLTK downloads on macOS, an SSL error appeared:

```
certificate verify failed: unable to get local issuer certificate
```

Cause: Python installed on macOS does not automatically use the system's trusted certificate store. It needs its own certificate bundle.

Short-term workaround (used in notebooks):

```
import ssl
ssl._create_default_https_context = ssl._create_unverified_context
```

Proper long-term fix (installed once):

```
open "/Applications/Python 3.14/Install Certificates.command"
```

This installs the certificate bundle and creates a symlink so Python can verify SSL connections properly. After this, the workaround is not needed.

4. Task 3 — Databases

Task 3 stored the processed Slack data in two different database systems. Using two databases was a deliberate architectural decision based on the different strengths of each type.

4.1 Why Two Databases?

	MongoDB (NoSQL)	PostgreSQL (SQL)
Type	Document store	Relational database
Structure	Flexible — each document can have different fields	Fixed schema — all rows same columns
Best for	Raw, varied, semi-structured data	Structured data with complex queries
Query language	MongoDB Query Language (JSON-like)	SQL (SELECT, JOIN, GROUP BY)
Our use	Raw messages, replies, reactions	Aggregated features and analytics

The analogy: MongoDB is like a filing cabinet where each folder can contain different types of documents. PostgreSQL is like a spreadsheet where every row must have exactly the same columns.

4.2 MongoDB — Collections and Documents

MongoDB organizes data as: Database → Collections → Documents. A Document is a JSON-like object. A Collection is a group of related documents.

Collection	Documents stored	Key fields
messages	20,011	channel, user, text, ts, type, reply_count
replies	8,900	channel, parent_ts, user, ts
reactions	5,017	channel, message_ts, name, count

```
# Connect to local MongoDB
from pymongo import MongoClient
client = MongoClient('mongodb://localhost:27017')
db = client['gebeya_slack']

# Insert documents
db['messages'].insert_many(message_docs)

# Query documents
results = db['messages'].find({'channel': 'ai-community-building'})
```

4.3 PostgreSQL — Tables and Rows

PostgreSQL stores data in tables with fixed columns. We created 3 tables for analytical queries:

Table	Rows	Purpose
message_features	19,602	One row per message with computed features
user_features	1,205	Aggregated stats per user across all messages

daily_sentiment	104	Average sentiment score per day
-----------------	-----	---------------------------------

```
# Connect to PostgreSQL
import psycopg2
conn = psycopg2.connect(
    dbname='gebeya_slack', user='btsm', host='localhost'
)

# Query with SQL
cur.execute('SELECT user, COUNT(*) FROM message_features GROUP BY user')
```

4.4 Indexing in MongoDB

An index in a database is like the index at the back of a textbook — instead of reading every page to find 'pandas', you go straight to the page number listed in the index. Indexes make queries dramatically faster.

However, not every field needs an index. Indexes cost storage space and slow down inserts (because the index must be updated too). Best practice: index fields you frequently filter or sort by, not everything.

```
# Create an index on channel field
db['messages'].create_index([('channel', 1)])

# Unique index – no two documents can have same value
db['messages'].create_index([('ts', 1)], unique=True)
```

5. Task 4 — Streamlit Dashboard

Task 4 built an interactive web dashboard to display all the analysis results visually. The dashboard was built with Streamlit — a Python framework that turns Python scripts into web apps without writing any HTML or JavaScript.

5.1 What is Streamlit?

Streamlit turns a Python script into a web page. Every time the user interacts (changes a dropdown, moves a slider), Streamlit re-runs the entire script from top to bottom and updates the page. This is why it's so simple to use — there's no complex event handling code needed.

```
import streamlit as st

st.title('My Dashboard')          # shows a title
st.metric('Messages', '20,011')   # shows a metric card
st.pyplot(fig)                    # shows a matplotlib chart
st.dataframe(df)                  # shows a scrollable table
channel = st.selectbox('Filter', channels) # dropdown
```

5.2 Dashboard Sections

Section	What it shows
1. Key Metrics	4 metric cards: total messages, channels, users, reactions
2. Top & Bottom Users	4 tabs — bar charts by messages, replies, reactions, mentions
3. Channel Activity	Scatter plot: messages vs engagement per channel
4. Message Classification	Bar chart + pie chart of message types
5. ML Analysis Results	Embedded PNG charts from Task 2 notebooks
6. Raw Data Explorer	Scrollable table with slider to choose row count

5.3 st.cache_data

Loading 20,000 messages from the database every time someone moves a slider would be very slow. `@st.cache_data` solves this — it runs the function once, stores the result in memory, and returns the cached result for all future calls.

```
@st.cache_data
def load_data():
    loader = SlackDataLoader()
    df = loader.load_all_channels()
    return df

df = load_data() # only queries the database ONCE, then uses cache
```

6. Docker — Containerization

Docker packages your application and all its dependencies into a container — a self-contained unit that runs identically on any machine. The analogy: a shipping container. You load cargo (your app + all dependencies) into a standardized container. Any ship (any computer) can carry it.

6.1 The Problem Docker Solves

Without Docker, running the dashboard on someone else's computer requires:

- Install Python 3.11 exactly
- Create a virtual environment
- Install all packages from requirements.txt
- Clone the repo with the right folder structure
- Run the correct startup command

With Docker, they just need Docker installed and one command:

```
docker run -p 8501:8501 -e MONGO_URI='...' btsm10/gebeya-dashboard
```

6.2 The Dockerfile Explained

A Dockerfile is a recipe for building a Docker image. Each instruction creates a layer:

```
FROM python:3.11-slim
# Base: minimal Linux + Python 3.11. 'slim' = stripped down, smaller size.

WORKDIR /app
# Create /app folder inside container and work from there.

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Install all packages. Done before copying code so Docker can cache this layer
.

COPY src/ ./src/
COPY app/ ./app/
# Copy only what's needed to run the dashboard.

EXPOSE 8501
# Open port 8501 so browser can reach the dashboard.

CMD ["streamlit", "run", "app/dashboard.py", "--server.port=8501"]
# Command that runs when container starts.
```

6.3 Docker Layers

Each Dockerfile instruction creates a layer. Layers are cached — if you change only app/dashboard.py, Docker only rebuilds the layers after COPY app/ — reusing the Python installation layer. This makes

rebuilds fast.

6.4 Docker Commands

Command	What it does
<code>docker build -t name .</code>	Build image from Dockerfile in current directory
<code>docker run -p 8501:8501 name</code>	Start container, map port 8501
<code>docker run -e VAR=value name</code>	Inject environment variable into container
<code>docker pull name:tag</code>	Download latest image from Docker Hub
<code>docker images</code>	List images stored on your machine
<code>docker ps</code>	List running containers
<code>docker stop ID</code>	Stop a running container

6.5 ARM vs AMD — The Platform Problem

Your Mac has an Apple Silicon chip (ARM64 architecture). GitHub Actions servers use Intel chips (AMD64 architecture). These are different instruction sets — a program compiled for one cannot run on the other without translation.

When the image was first built on GitHub Actions (AMD64) and run on your Mac (ARM64), Docker showed a platform mismatch warning and streamlit failed to start because the binary was compiled for the wrong architecture.

Fix: build the image for both platforms using Docker Buildx:

```
- name: Set up QEMU
  uses: docker/setup-qemu-action@v3

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push
  uses: docker/build-push-action@v5
  with:
    platforms: linux/amd64,linux/arm64
    tags: btsm10/gebeya-dashboard:latest
```

QEMU is an emulator that allows the AMD64 GitHub Actions server to compile code for ARM64. This produces a multi-platform image that works on Mac, Windows, and Linux.

6.6 Environment Variables in Docker

A container is isolated — it doesn't see your Mac's environment variables. The `-e` flag injects values from outside:

```
docker run -p 8501:8501 \
  -e MONGO_URI='mongodb+srv://user:pass@cluster.mongodb.net/' \
```

```
btsm10/gebeya-dashboard
```

Inside the container, `os.environ.get('MONGO_URI')` finds this value and the dashboard connects to Atlas. Without `-e`, `MONGO_URI` is empty and the loader falls back to local files (which don't exist inside the container).

7. Task 5 — Infrastructure & CI/CD

Task 5 covered the infrastructure and automation layer — the systems that build, test, deploy, and host the application automatically.

7.1 GitHub Actions — CI/CD Pipeline

CI/CD stands for Continuous Integration / Continuous Deployment. Every time you push code to GitHub, a series of automated jobs run:

- CI (Continuous Integration): automatically test and validate new code
- CD (Continuous Deployment): automatically build and deploy if tests pass

The complete pipeline on every push to main:

```
git push origin main
|
■■■ Flake8 Lint (9s)      → checks code style
■■■ Docstring Tests (31s) → checks documentation
■■■ Unit Tests (85s)     → runs pytest with MONGO_URI
■■■ Build & Push Docker  → builds multi-platform image → Docker Hub
```

7.2 GitHub Secrets

Secrets are encrypted values stored in GitHub that your workflows can access but nobody can read — not even you can see them after saving. They are never exposed in logs or code.

Secret name	Used by	Value
MONGO_URI	Unit Tests workflow	MongoDB Atlas connection string
DOCKER_USERNAME	Docker publish workflow	btsm10
DOCKER_PASSWORD	Docker publish workflow	Docker Hub password

Secrets are accessed in workflows using double curly brace syntax:

```
env:
  MONGO_URI: ${ secrets.MONGO_URI }
```

7.3 Terraform — Infrastructure as Code

Terraform is a tool that lets you define cloud infrastructure in code files instead of clicking through web consoles. The entire infrastructure is described in .tf files and can be created or destroyed with commands.

Command	What it does
terraform init	Download provider plugins (AWS, etc.)
terraform validate	Check syntax of .tf files
terraform plan	Preview what would be created/changed/destroyed

terraform apply	Actually create the infrastructure on AWS
terraform destroy	Destroy everything (careful — permanent)

Our terraform/main.tf creates two AWS resources:

- EC2 t2.micro instance — a small virtual Linux server to run the app
- S3 bucket — cloud file storage (like Google Drive but for code/data)

Terraform is validated and ready. Applying it requires an AWS account. AWS has a 12-month free tier that covers t2.micro and basic S3 usage, but requires a credit card to sign up.

7.4 What is AWS?

Amazon Web Services (AWS) is Amazon's cloud computing platform — the world's largest. Instead of buying physical servers, you rent virtual servers, storage, databases, and hundreds of other services on demand. You pay only for what you use.

AWS Service	What it is	Our use
EC2	Virtual server (Elastic Compute Cloud)	Run the Docker container
S3	File storage (Simple Storage Service)	Store data files and artifacts
IAM	Identity and Access Management	Control who can access what

8. MongoDB Atlas — Cloud Database

MongoDB Atlas is the cloud-hosted version of MongoDB. Instead of running MongoDB on your Mac (which only you can access), Atlas runs it on servers in the cloud that anyone with credentials can reach — including GitHub Actions.

8.1 Why We Migrated to Atlas

The GitHub Actions unit tests were failing because:

- Tests use SlackDataLoader which reads from anonymized/ folder
- anonymized/ is in .gitignore — never pushed to GitHub for privacy
- GitHub Actions clones the repo — no anonymized/ folder
- Tests crash: FileNotFoundError

The solution: move the data to Atlas. GitHub Actions connects to Atlas using the MONGO_URI secret, gets the data, tests pass.

8.2 Atlas Setup Steps

Step	Action
1	Create free account at mongodb.com/atlas
2	Create free cluster (M0 Sandbox — 512MB, free forever)
3	Create database user: biniamtsiget_db_user
4	Network Access: allow 0.0.0.0/0 (any IP — needed for GitHub Actions)
5	Get connection string from Connect → Drivers → Python
6	Run migration script to load data from JSON files

8.3 The Migration Script

Instead of migrating from local MongoDB (which had no data), we loaded directly from the raw JSON files in anonymized/ into Atlas:

```
# migrate_to_atlas.py
client = MongoClient(ATLAS_URI)
db = client['gebeya_slack']

# Read JSON files
for channel in os.listdir('anonymized/'):
    for json_file in channel_folder:
        messages.append({...})

# Upload to Atlas
db['messages'].drop() # clear old data
db['messages'].insert_many(messages) # 20,011 documents
```

Result: 20,011 messages, 8,900 replies, 5,017 reactions loaded to Atlas.

8.4 SSL Certificate Fix for Atlas Connection

When first connecting to Atlas, an SSL error appeared:

```
pymongo.errors.ServerSelectionTimeoutError: SSL: CERTIFICATE_VERIFY_FAILED
```

Same root cause as the NLTK SSL issue — Python on macOS doesn't automatically use the system certificate store. Fixed by running the certificate installer:

```
open "/Applications/Python 3.14/Install Certificates.command"
```

This installed the certifi bundle and created a symlink. After this, pymongo can verify Atlas's SSL certificate and connect successfully.

8.5 How SlackDataLoader Uses Atlas

The loader was updated to automatically detect whether Atlas is available:

```
# src/config.py
try:
    import streamlit as st
    MONGO_URI = st.secrets.get('MONGO_URI', os.environ.get('MONGO_URI', ''))
except Exception:
    MONGO_URI = os.environ.get('MONGO_URI', '')

# src/loader.py
def __init__(self):
    self.source = 'mongo' if MONGO_URI else 'local' # auto-detect
```

Environment	MONGO_URI set?	Data source
Local Mac (no env var)	No	Local JSON files in anonymized/
GitHub Actions	Yes (secret)	MongoDB Atlas
Docker container	Yes (-e flag)	MongoDB Atlas
Streamlit Cloud	Yes (st.secrets)	MongoDB Atlas

9. Streamlit Cloud — Public Deployment

Streamlit Cloud (share.streamlit.io) is a free hosting service for Streamlit apps. It deploys directly from your GitHub repository — no servers to manage, no Docker to configure. Anyone with the URL can open the dashboard.

9.1 How It Works

- Connect your GitHub account to Streamlit Cloud
- Select your repository, branch, and the main Python file
- Streamlit Cloud clones your repo and installs `requirements.txt`
- Your app runs on Streamlit's servers and gets a public URL
- Every git push to main automatically redeploys the app

9.2 Streamlit Secrets

Streamlit Cloud has its own secrets manager — separate from GitHub Secrets. You add secrets in the Advanced Settings before deploying. They are stored encrypted on Streamlit's servers and injected via `st.secrets`:

```
# In Streamlit Cloud secrets (TOML format):
MONGO_URI = 'mongodb+srv://...'
```

```
# In Python code:
import streamlit as st
uri = st.secrets['MONGO_URI']
```

9.3 Streamlit vs Docker — When to Use Each

	Streamlit Cloud	Docker
Best for	Sharing dashboards publicly, demos	Production apps, any language
Setup	Connect GitHub, click deploy	Write Dockerfile, manage registry
Cost	Free for public repos	Free image, hosting costs extra
Access	Public URL — anyone can open	Need to run <code>docker run</code> command
Auto-redeploy	Yes — on every git push	Only if CI/CD configured
Data privacy	Code is visible on GitHub	Image on Docker Hub, data separate

9.4 Issues Encountered

Two errors were hit during Streamlit Cloud deployment:

Error 1: `FileNotFoundError`

```
d for d in os.listdir(self.data_path)
```

Cause: MONGO_URI wasn't being read correctly. Streamlit uses st.secrets, not os.environ. Fix: updated config.py to try st.secrets first.

Error 2: KeyError 'reactions'

```
df['reaction_count_clean'] = df['reactions'].apply(...)
```

Cause: MongoDB messages collection doesn't have a 'reactions' column — we only stored reply_count during migration. Fix: added column existence check.

```
df['reaction_count_clean'] = (  
    df['reactions'].apply(...) if 'reactions' in df.columns else 0  
)
```

10. All Mistakes Made & How They Were Fixed

Every error encountered during the project, what caused it, and how it was fixed. Understanding mistakes is as important as understanding the solutions.

Error	Cause	Fix
gensim install failed (build wheel error)	gensim requires C++ compiler and is hard to build on new Python	Switched to new PyTorchDA — same algorithm, easier install
ModuleNotFoundError: nltk (in Jupyter)	Jupyter was using system Python kernel, not the new one	Registered new env as Jupyter kernel with ipykernel install
SSL: CERTIFICATE_VERIFY_FAILED (NLTK download)	macOS Python doesn't use system certificates	Added ssl._create_unverified_context workaround in notebook
SSL: CERTIFICATE_VERIFY_FAILED (MongoDB Atlas)	Same SSL issue, pymongo can't verify Atlas certificate	Run certifi.Certificates.command — proper permanent fix
terraform not found (after brew install)	brew install terraform uses wrong formula	Used brew install hashicorp/tap/terraform instead
.terraform/ committed to git	Large provider binaries accidentally staged	git rm -r --cached .terraform/ and added to .gitignore
Flake8 CI failing	Unused imports (pd, np), E402 warnings, line length	Removed unused imports, added # noqa: E402, excluded generated files
pytest: unused import pytest (F401)	import pytest was in test file but pytest is a module	Removed the unused import line
migrate_to_atlas: no documents	Local MongoDB had no data — gebeya_slack_db was empty	Revised migration to load directly from JSON files instead
Migration: type field missing	Migration script stored 5 fields but forgot type field	Added type field to migration, re-ran migration
Docker: streamlit not found	streamlit and Pillow were missing from requirements.txt	Added both to requirements.txt
Docker: platform mismatch (ARM vs AMD)	GitHub Actions builds AMD64, Mac is ARM64	Updated workflow to build linux/amd64,linux/arm64 with buildx
Docker: KeyError reactions	reactions column not in MongoDB message collection	Added collection existence check before accessing reactions
Streamlit: FileNotFoundError	Config.py used os.environ but Streamlit uses st.secrets	Updated config.py to check st.secrets first, then os.environ
pymongo not found (GitHub Actions)	pymongo missing from requirements.txt	Added pymongo to requirements.txt

11. Questions Asked & Answers

Every conceptual question asked during the project, with detailed answers.

Q: What is gensim and why was it removed?

Gensim is a natural language processing library that includes LDA topic modelling. It was initially planned for topic modelling but failed to install because it requires compiling C++ extensions, which is complex on newer Python versions. We switched to scikit-learn's LDA implementation which is pure Python and installs easily.

Q: What is sentiment analysis?

Sentiment analysis determines the emotional tone of text — positive, negative, or neutral. TextBlob assigns a polarity score from -1.0 (very negative) to +1.0 (very positive). We used it to measure how the mood of the Gebeya Slack community changed over time.

Q: Have we trained the model so far?

Yes — the LDA topic model was the only genuine model training. Calling `lda.fit(X)` trains the model on our Slack messages. TextBlob sentiment analysis used a pre-trained model (no training done by us). Message classification was rule-based (no ML at all).

Q: What is Homebrew?

Homebrew is the package manager for macOS system tools — like an App Store for developer software. You use it to install MongoDB, PostgreSQL, Terraform, etc. It's different from pip: Homebrew installs system tools globally, pip installs Python libraries inside a virtual environment.

Q: Where on my Mac do I find Homebrew?

Homebrew installs tools to `/opt/homebrew/` on Apple Silicon Macs. You don't find it as an app — you use it entirely from the terminal. Type `'brew list'` to see everything installed, `'brew services list'` to see running services.

Q: Does Homebrew work for both global and venv?

Homebrew installs things globally on your Mac — available everywhere. Virtual environments (gebvsnv) only contain Python packages installed with pip. They're separate systems: Homebrew for system tools, pip for Python libraries.

Q: What happens when I save a Jupyter notebook?

The `.ipynb` file saves everything: your code cells, output (charts, tables, printed text), and metadata. It's a JSON file. All outputs are embedded inside it — you don't need to run the code again to see previous results.

Q: What does `source` do?

`'source'` runs a shell script in the current terminal session. `'source gebvsnv/bin/activate'` runs the activation script which modifies your `PATH` so Python points to gebvsnv's Python instead of system Python.

Q: What is `__pycache__`?

`__pycache__` contains compiled versions of your Python files (`.pyc` files). Python compiles your source code to bytecode the first time it runs, then reuses the cached version for faster subsequent runs. It's auto-managed — you never need to touch it.

Q: Doesn't `__pycache__` make Python a compiled language?

No — Python is still interpreted. The .pyc bytecode still needs the Python interpreter to run. It's just a performance optimization. True compiled languages (like C or Go) produce native machine code that runs without any interpreter.

Q: Why did we use two databases?

MongoDB (NoSQL) is flexible — perfect for raw, varied Slack JSON data where each message can have different fields. PostgreSQL (SQL) is structured — perfect for analytical queries with GROUP BY, aggregations, and joins. They serve different purposes: MongoDB for storage, PostgreSQL for analytics.

Q: Is it a good idea to index everything in MongoDB?

No. Indexes speed up reads but slow down writes and use extra storage. Only index fields you frequently filter or sort by. Over-indexing is a common mistake that wastes resources.

Q: What is Terraform?

Terraform is Infrastructure as Code — instead of clicking through AWS console to create servers and storage, you write .tf files describing what you want and Terraform creates it. This makes infrastructure reproducible, version-controlled, and easy to recreate after destroying.

Q: What is Docker?

Docker packages your app and all its dependencies into a container — a self-contained unit that runs identically on any machine. The problem it solves: 'it works on my machine but not yours'. With Docker, if it works on your Mac, it works on the server.

Q: What is AWS?

Amazon Web Services is the world's largest cloud computing platform. Instead of buying physical servers, you rent virtual ones on demand. EC2 is their virtual server service, S3 is file storage. You pay only for what you use, and there's a 12-month free tier for new accounts.

Q: Can someone who has my repo run the dashboard on their device?

Yes — they need Docker installed and one command: `docker run -p 8501:8501 -e MONGO_URI='your_atlas_string' btsm10/gebeya-dashboard`. Docker downloads the image from Docker Hub and runs it. No Python, no pip, no virtual environments needed.

Q: What are those layers when pulling from Docker Hub?

Each Docker instruction in the Dockerfile creates a layer. When you pull an image, each layer downloads separately. They're shown as hashes. If you pull an updated image, Docker only downloads changed layers — reusing cached unchanged layers. This makes updates much faster.

Q: If there's no change to Docker and change is only in other files, should Docker be downloaded again?

No — if the code inside the container didn't change, pulling again gets you the exact same image. In a production setup, you'd configure the Docker workflow to only trigger when `src/`, `app/`, `requirements.txt`, or Dockerfile changes. For now, our workflow rebuilds on every push to main.

Q: Is `SlackDataLoader` a class?

Yes. It's defined with `'class SlackDataLoader:'` and has three methods: `get_channels()`, `load_channel()`, and `load_all_channels()`. Using a class means `data_path` is set once in `__init__` and all methods share it via `self`.

Q: What does the MONGO_URI environment variable do?

`os.environ.get('MONGO_URI', '')` reads a value stored in the operating system's environment. If set, `SlackDataLoader` connects to MongoDB Atlas. If not set, it falls back to reading local JSON files. This allows the same code to work in all environments without any changes.

Q: What are docstrings and unit tests in GitHub Actions?

Docstrings are triple-quoted descriptions inside functions: `"""Does X."""` The docstring workflow checks all functions have them. Unit tests are small code snippets that verify one function works correctly. The unit test workflow runs `pytest` tests/ on every push. Together they enforce code quality automatically.

Q: Is GitHub safe to give access to Streamlit?

Yes. GitHub Secrets (`MONGO_URI`, `DOCKER_PASSWORD`) are encrypted and only accessible to GitHub Actions — Streamlit cannot see them. Streamlit only reads your code files. The `MONGO_URI` for Streamlit is stored separately in Streamlit's own secrets manager, completely independent from GitHub Secrets.

12. Final System Architecture

The complete pipeline from raw data to publicly accessible dashboard:

```
Raw Slack JSON files (anonymized/)
|
v
migrate_to_atlas.py
|
v
MongoDB Atlas (cloud)
gebeya_slack database
3 collections: messages, replies, reactions
|
v
SlackDataLoader (src/loader.py)
Auto-detects: Atlas if MONGO_URI set, else local JSON
|
v
Streamlit Dashboard (app/dashboard.py)
Packaged in Docker image: btsm10/gebeya-dashboard
|
■■■> Docker Hub (btsm10/gebeya-dashboard:latest)
|       Anyone: docker run -p 8501:8501 ...
|
■■■> Streamlit Cloud (share.streamlit.io)
|       Public URL — no setup needed
```

Component	Technology	Status
Data storage	MongoDB Atlas (cloud)	Live — 20,011 messages
Data loading	SlackDataLoader class	Auto-detects Atlas or local
Analysis	pandas, sklearn, TextBlob, MLFlow	Complete
Dashboard	Streamlit	Running on Streamlit Cloud
Containerization	Docker (multi-platform)	Image on Docker Hub
Code style CI	Flake8 GitHub Actions	Passing
Documentation CI	Docstring checker	Passing
Testing CI	pytest + MongoDB Atlas	All 5 tests passing
Cloud infrastructure	Terraform (AWS EC2 + S3)	Ready — needs AWS account

This document covers the complete Gebeya Week 1 Challenge — from raw JSON files to a live public dashboard with a full CI/CD pipeline. Every tool used, every concept explained, every mistake fixed.